CSC490 A2

Data Processing Pipelines and Infrastructure-as-Code

Avni Agrawal (agraw264)
Chau Nguyen (nguy2829)
Rishit Dagli (dagliris)

Contents

1 Aspirational Datasets **3**

1.1 Training and Inference Pipeline Overview . . . 3

1.2 Aspirational Dataset Design . . . 4

1.3 Dataset Schema . . . 4

2 Reality Check **6**

3 Data-processing pipelines **8**

3.1 Data lake design . . . 9

3.2 Pipeline stages (Bronze, Silver, Gold) . . . 10

3.2.1 Bronze Layer: Fault-Tolerant Ingestion . . . 10

3.2.2 Silver Layer: Transformation . . . 10

3.2.3 Gold Layer: Aggregation and dataset manifest . . . 11

3.3 Data schemas . . . 11

3.4 Pipeline Execution and Use Cases . . . 12

3.5 Initial implementation . . . 12

3.6 Next Steps and Future Features . . . 13

4 Infrastructure as Code Implementation **13**

4.1 Infrastructure Completeness and Modular Design 13

4.2 How IaC aligns with the pipeline 14

4.3 Implementation of Multiple Environments 15

5 Disaster Recovery Demonstration **15**

1 Aspirational Datasets

1.1 Training and Inference Pipeline Overview

Figure 1 shows the training and inference pipeline of Compos3D. The central objective of the framework is to learn a strong set of interpretable hypotheses about how valid 3D scenes should be constructed. Instead of directly predicting geometry, the model learns reusable design rules in the form of Blender-executable graphs that guide structured scene generation.

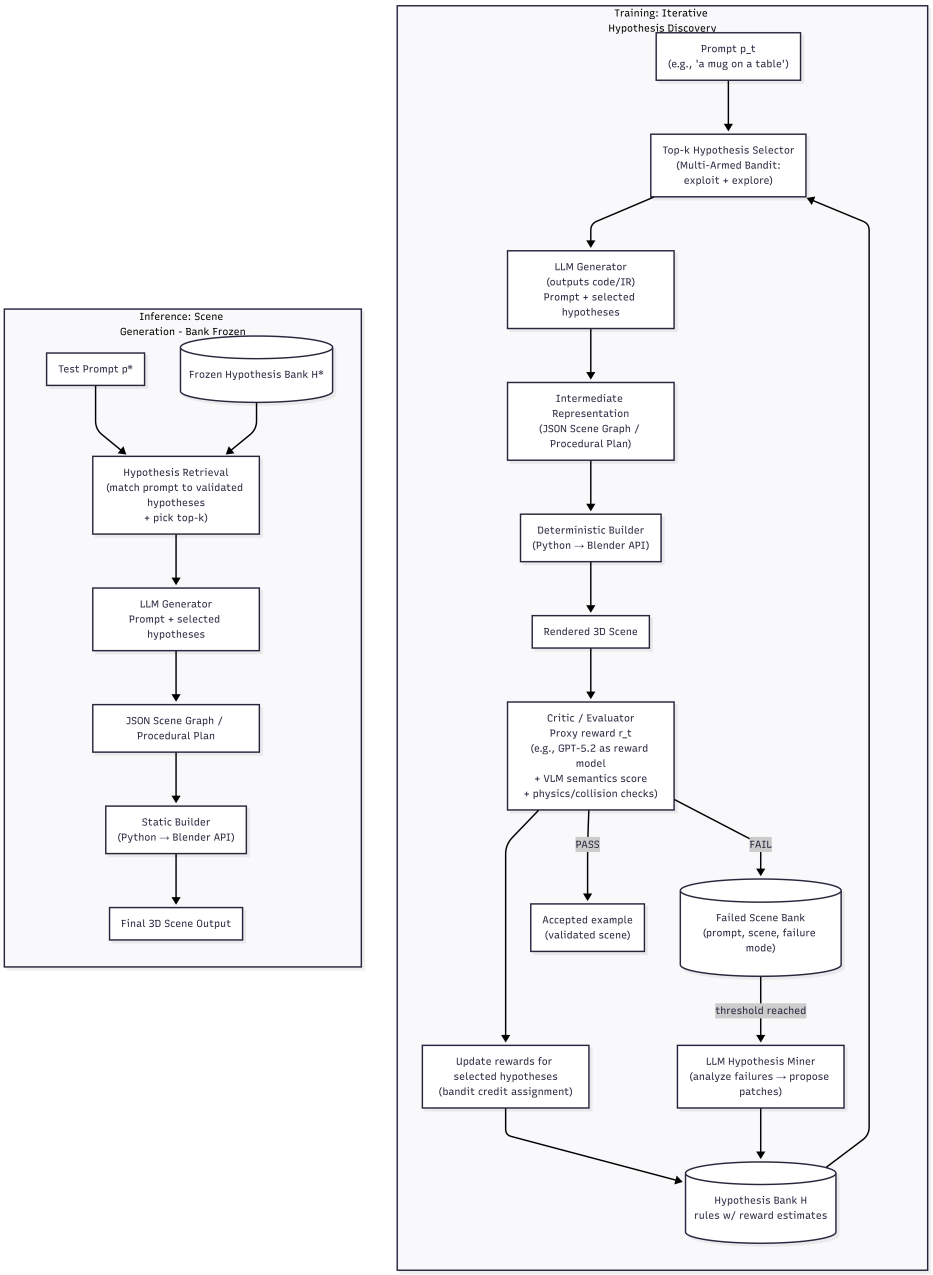

Figure 1: Training and inference pipeline of Compos3D.

During **training**, the system operates in an iterative hypothesis discovery loop. For each prompt, a subset of candidate hypotheses is selected from a hypothesis bank using a bandit-style strategy that balances exploration and exploitation. An LLM generator conditions on the prompt and selected hypotheses to produce structured scene-editing code or an intermediate scene graph. This representation is executed by a deterministic builder that constructs and renders a 3D scene in Blender. A critic model then evaluates the generated scene and produces a proxy reward based on physics plausibility, semantic alignment with the prompt, and structural correctness. These rewards are attributed back to the selected hypotheses, updating their utility estimates. Failed scenes are accumulated in a failure bank, and an LLM analyzes recurring failure modes to propose new hypotheses that patch missing design rules.

During **inference**, the hypothesis bank is frozen. Given a new prompt, the system retrieves the most relevant hypotheses and uses them to guide the generator in producing structured scene-editing code. The builder executes this code to produce the final 3D scene.

1.2 Aspirational Dataset Design

To support the Compos3D pipeline, we propose a dataset of structured 3D scenes paired with natural language prompts, human rationales, and lightweight annotations about scene composition and realism. Each entry links a prompt to a 3D scene and short human explanations describing why the scene is plausible or implausible.

Aspirationally, we focus on multi-object 3D scenes composed of artist-designed assets that may include textures. Each scene is paired with a natural language prompt and human rationales explaining why the arrangement is plausible or implausible. By emphasizing multiple interacting objects and textured, artist-created assets, the dataset enables hypotheses such as stability, support, and spatial relations to be evaluated under progressively more complex and realistic settings.

1.3 Dataset Schema

i. Core Entry Schema

```
{
  "scene_id": str,
  "prompt": str,

  "scene_3d": {
    "representation": "mesh | gaussian_splat | radiance_field",
    "textured": bool,
    "asset_paths": [str]
  },

  "objects": [
    {"name": str, "count": int}
  ],
```

```
  "scene_graph": {
    "nodes": [str],
    "edges": [{"relation": str, "from": str, "to": str}]
  },

  "human_rationales": [str],
  "physical_constraints": [str],

  "scores": {
    "semantic_plausibility": float,
    "physical_validity": float,
    "compositional_coherence": float
  },

  "metadata": {
    "num_objects": int,
    "realism_level": "cartoon | realistic",
    "has_texture": bool,
    "interaction_complexity": "low | medium | high"
  }
}
```

- **Prompt**: A natural language description specifying the intended multi-object scene and interactions among objects.
- **3D Scene**: Each scene is constructed from artist-designed assets that can include textures, enabling visually rich and compositional environments that resemble real-world arrangements while remaining controllable.
- **Objects and Scene Graph**: The dataset enumerates multiple objects and encodes their spatial and functional relations (e.g., support, adjacency, containment), which are essential for learning compositional structure and interaction rules.
- **Human Rationales**: Annotators provide concise explanations describing why the multi-object arrangement is plausible, directly linking rationales to object interactions such as support, stability, and spatial layout.
- **Physical Constraints**: Explicit constraints describe simple physical rules (e.g., objects must be supported, avoid collisions), grounding the scenes in physically valid configurations.
- **Scores** Each scene is annotated with numeric scores that provide quantitative supervision for evaluating generated scenes:
  - **Semantic Plausibility**: Measures how well the scene matches the prompt and expected object semantics (e.g., correct object types and roles).
  - **Physical Validity**: Captures basic physical correctness such as support, stability, and absence of collisions or floating objects.
  - **Compositional Coherence**: Evaluates whether spatial relations among multiple objects are consistent and form a logically arranged scene.

ii. Global Hypothesis Bank Schema

```
{
  "hypothesis": str,
  "supporting_scenes": [str],
  "confidence": float,
  "source": "human | discovered | refined"
}
```

The global hypothesis bank stores reusable scene rules derived from human rationales and observed across many multi-object scenes. Each rule is linked to the scenes that support it and assigned a confidence score, allowing the training pipeline to select, test, and refine rules that improve the plausibility and coherence of generated scenes.

2 Reality Check

The following table summarizes existing public datasets, datasets that can be scraped, and datasets that can be generated to approximate the aspirational Compos3D dataset.

Available datasets and data sources relevant to the Compos3D project.			
Dataset	**Link**	**Description**	**Commentary on Relevance**
BlenderBench	https://huggingface.co/datasets/DietCoke4671/BlenderBench	A benchmark dataset of Blender scenes with natural language tasks, rendered images, and executable Python code for iterative scene editing.	Provides paired start/goal scenes with executable Blender code and renders, which aligns with our need for structured scene transformations. However, it doesn't have explicit human rationales, physical constraints, and metadata about object interactions and realism level required by the ideal schema.
Objaverse-XL	https://objaverse.allenai.org/	A large-scale open dataset containing millions of annotated 3D objects across diverse real-world categories.	Offers millions of diverse, textured 3D objects from varied sources, closely matching the schema's requirement for artist-designed assets with textures. However, it only contains individual objects rather than multi-object scenes with scene graphs, rationales, and physical validity annotations.

Available datasets and data sources relevant to the Compos3D project.			
Dataset	**Link**	**Description**	**Commentary on Relevance**
HM3D (Habitat-Matterport 3D)	https://arxiv.org/abs/2109.08238	A dataset of large-scale real-world indoor 3D environment reconstructions.	Provides large-scale, photo-realistic multi-object indoor scenes with realistic layouts, supporting complex object interactions in the schema. However, it consists of real-world scans and lacks explicit human rationales, editable object-level scene graphs, and controllable artist-designed assets.
OmniObject3D	https://arxiv.org/abs/2301.07525	A real-scanned dataset of everyday objects with meshes and multi-view images.	Contains high-quality real scanned objects with textures and multi-view data, matching the schema's need for realistic object geometry and appearance. However it focuses on isolated objects rather than compositional multi-object scenes with relational graphs and rationales.
ShapeNet	https://shapenet.org	A large repository of categorized 3D CAD models across thousands of classes.	Provides large-scale categorized 3D CAD objects useful for object-level semantics in the schema. However, it lacks textures, multi-object scene composition, and annotations for spatial relations or human plausibility rationales.
Google Scanned Objects	https://research.google/resources/datasets/google-scanned-objects/	High-quality real-world scanned objects with accurate textures and materials.	Includes clean, realistic scanned objects with accurate materials, aligning with the schema's textured asset requirement. However, it lacks multi-object scene arrangements, scene graphs, and rationale-linked physical constraints.

Available datasets and data sources relevant to the Compos3D project.			
Dataset	**Link**	**Description**	**Commentary on Relevance**
DL3DV-10K	https://dl3dv-10k.github.io/DL3DV-10K/	A large-scale dataset of real-world videos reconstructed into 3D scenes.	Contains large-scale multi-view 3D video data suitable for reconstruction and view consistency, partially supporting scene realism. However, it does not provide structured scene graphs, object counts, or human rationales required for hypothesis learning.
(Generated dataset) Scraped Multi-View Videos + NeRF / Gaussian Splatting	N/A	Multi-view real-world videos reconstructed into 3D scenes using NeRF or Gaussian Splatting pipelines.	Can produce realistic multi-object 3D scenes with textures that match the schema's scene representation and realism goals. However, they typically lack structured object annotations, scene graphs, and human rationales, and require substantial effort to curate and standardize.
(Generated dataset) Procedural Blender Scene Generation	N/A	Synthetic scenes generated programmatically in Blender with controlled object placement and relations.	Allows precise control over object count, relations, and textures, directly matching the schema's need for structured multi-object scenes and physical constraints. The drawback is the significant engineering effort required to design, generate, and annotate large-scale scenes compared to leveraging existing datasets.

3 Data-processing pipelines

Our goal with the data pipelines is to turn raw BlenderBench [2] scenes into curated, training-oriented datasets. The pipeline is implemented in Python and executed as Ray [1] tasks on Anyscale; data are persisted in an S3-backed medallion layout (Bronze, Silver, Gold). This design intends to (i) isolate failure modes between ingestion and transformation, (ii) make intermediate artifacts inspectable, and (iii) support repeated re-processing as transformation logic evolves.

Table 2: Core components

Component	Technology
Distributed compute	Ray on Anyscale
Data lake storage	Amazon S3 (3 buckets per env: Bronze/Silver/Gold)
Catalog + SQL query	AWS Glue Data Catalog + Glue Crawler; AWS Athena workgroup
Secrets	AWS Secrets Manager secret containers (API keys)
Primary data source	HuggingFace Datasets (`DietCoke4671/BlenderBench`)
Data formats	JSON and Parquet

Figure 2: An overview of the architecture used by Compos3D for data. Each layer is written to its own S3 bucket (per environment) under a shared prefix, so intermediate artifacts remain visible and reprocessable.

3.1 Data lake design

The pipeline writes all artifacts into S3 using three buckets (Bronze/Silver/Gold) and a common prefix (`compos3d`). The repository implements bucket-per-layer storage because it makes access control and lifecycle policy reasoning straightforward (e.g., Bronze can be treated as an immutable landing area, while Gold is consumed by training jobs).

Within each bucket, objects are partitioned by UTC date and scene identifier:

```
s3://<bronze_bucket>/compos3d/bronze/scenes/YYYY/MM/DD/scene_<id>/scene.json
s3://<silver_bucket>/compos3d/silver/scenes/YYYY/MM/DD/scene_<id>/scene.json
s3://<gold_bucket>/compos3d/gold/training_datasets/YYYY/MM/DD/dataset.json
```

This partitioning matches how the code enumerates work: Silver lists `bronze/scenes/` and trans-

forms each `scene.json`; Gold lists `silver/scenes/` and aggregates all transformed scenes into a single dataset manifest.

3.2 Pipeline stages (Bronze, Silver, Gold)

The core design decision is to decouple ingestion, transformation, and aggregation into separate stages (Figure 2). In practice, this allows the pipeline to re-run Silver/Gold without re-downloading Blender-Bench, and it keeps stage-specific logic simple and testable.

3.2.1 Bronze Layer: Fault-Tolerant Ingestion

Bronze ingests a configurable number of BlenderBench instances. Each instance is processed by a Ray remote function, which downloads the dataset metadata, fetches the corresponding Blender file into a worker-local cache, and writes a Bronze JSON record to S3.

We handle all failures per-instance: if a download fails or an instance is missing, the Ray task returns `success=False` and the driver prints the error while continuing to process remaining instances. This is intentional, ingestion from the external source can be flaky, and it is better to make partial progress than to fail an entire run due to a small number of outliers.

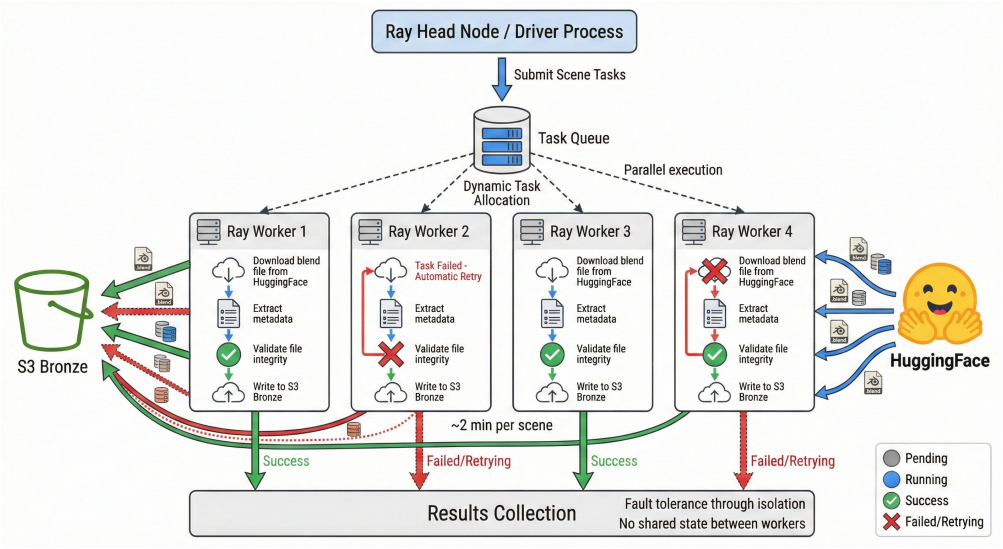

Figure 3: Bronze ingestion. The driver submits per-instance tasks to Ray; workers independently download instances and write Bronze records to S3.

3.2.2 Silver Layer: Transformation

Silver is a deterministic transformation of Bronze records. The driver enumerates all Bronze `scene.json` objects under `bronze/scenes/` (optionally filtered by date), and Ray tasks read each Bronze record and write a corresponding Silver record.

In the current implementation, the Silver record is still JSON: it normalizes a small set of fields and computes derived attributes used later for aggregation (e.g., numeric complexity level derived from `level`, code length statistics, and a coarse `task_type` categorization). This is sufficient to demonstrate ingestion → cleaning/transformation → aggregation end-to-end, while keeping artifacts easy to inspect. A Parquet upgrade is explicitly planned as a next step.

3.2.3 Gold Layer: Aggregation and dataset manifest

Gold aggregates the entire set of Silver scenes into a training dataset manifest (`dataset.json`). The driver lists all Silver `scene.json` objects under `silver/scenes/`, batches them, and uses Ray tasks to compute batch statistics. Results are merged on the driver into a single dataset document that includes:

- counts and distributions (by level and by task type),
- summary code-length statistics,
- a compact list of scenes (IDs + a few key fields).

The manifest is written to the Gold bucket under `gold/training_datasets/YYYY/MM/DD/`.

3.3 Data schemas

The tables below reflect the exact JSON fields from the BlenderBench dataset written by the current pipeline implementation. These schemas are intentionally small: they capture enough structure for quality checks and dataset aggregation, while staying easy to evolve.

Table 3: Schemas by layer

Layer	Format	Key Fields
Bronze	JSON	`scene_id`, `source`, `instance_id`, `level`, `task_description`, `blend_file_path`, `blend_file_size_mb`, `start_code`, `goal_code`, `ingestion_timestamp`
Silver	JSON	`scene_id`, `source`, `level`, `task`, `complexity_level`, `code_length_start`, `code_length_goal`, `task_type`, `has_camera`, `has_attribute`, `blend_file`, `blend_file_size_mb`, `ingested_at`, `transformed_at`
Gold	JSON	`dataset_id`, `created_at`, `num_scenes`, `statistics` (e.g., `by_level`, `by_task_type`, `avg_code_length`), `scenes` (list of `scene_id`, `level`, `task_type`, `complexity`)

3.4 Pipeline Execution and Use Cases

The pipeline execution flow (Figure 4) supports iterative development and repeatable dataset generation.

- **Development runs**: run on a small sample size to validate end-to-end behavior quickly.
- **Dataset refresh runs**: re-run Bronze to ingest a larger slice of BlenderBench, then re-run Silver/-Gold to regenerate the transformed and aggregated artifacts.
- **Downstream consumption**: training code consumes the Gold `dataset.json` manifest to select scenes and inspect dataset-level statistics.

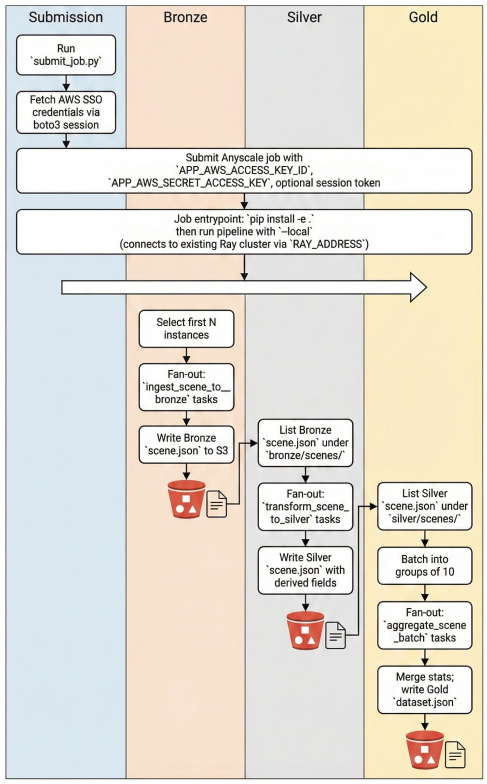

Figure 4: Our data processing pipelines. Each stage processes only successfully completed scenes from the previous stage.

3.5 Initial implementation

All of our code for this section is present in our repository: `https://github.com/UofT-CSC490-W2026/Compos3D` with clear instructions on setting up the dev environment, as well as dev, staging, and prod resources on AWS and Anyscale.

3.6 Next Steps and Future Features

Our current implementation has some areas of improvements.

- **Persist raw assets**: upload the `.blend` file (and/or extracted assets) to S3 in Bronze, and store its S3 URI in the Bronze/Silver records (currently the record stores a worker-local path).
- **Stronger validation**: add explicit schema validation and clearer error reporting in Silver (e.g., missing fields, malformed levels).
- **Parquet for Silver/Gold**: write Silver (and potentially Gold) as Parquet to improve query performance; keep a small JSON manifest for metadata.
- **Glue integration**: trigger the Glue crawler after Silver writes complete, so Athena tables stay up to date automatically.
- **Incremental runs**: maintain a lightweight watermark (e.g., under an S3 prefix) so Silver and Gold can process only new dates/partitions.

4 Infrastructure as Code Implementation

Compos3D's AWS infrastructure is provisioned with Terraform. The intent is to make storage and governance primitives reproducible across environments (`dev`, `staging`, `prod`) without manual console work, which would only be possible to implement in future assignments.

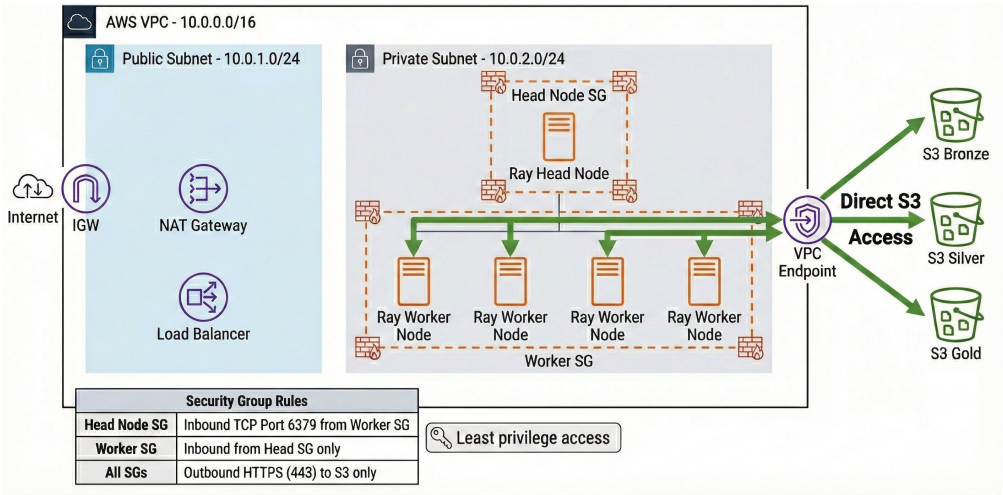

Figure 5: Architecture of VPC.

4.1 Infrastructure Completeness and Modular Design

The root module composes four modules that map directly to the pipeline's needs: storage (S3), catalog/-query (Glue + Athena), secrets (Secrets Manager), and IAM. Optional modules for ECR and AWS Batch

exist in the repository but are not enabled in the current root module.

Table 4: Terraform modules and what they provision

Module	Provisioned resources and purpose
`s3_lake`	Creates three S3 buckets (Bronze/Silver/Gold) with versioning enabled, default SSE-S3 encryption, and public access blocked. Bronze additionally has lifecycle transitions (to `STANDARD_IA` after 90 days, then `GLACIER_IR` after 180 days).
`glue_catalog`	Creates a Glue database and a Glue crawler that targets `s3://<silver_bucket>/compos3d/silver/`. Also creates an Athena workgroup with results stored under `s3://<silver_bucket>/athena-results/` with SSE-S3 enabled.
`secrets`	Creates Secrets Manager secret containers for API keys (OpenAI, Anthropic, Anyscale, W&B) and an IAM policy granting read access to those secrets.
`iam`	Creates IAM roles/policies used for querying and (future) batch execution: a Batch job role with S3/Glue access (and secrets-read attached), a Batch service role, and an Athena user role with S3/Glue/Athena permissions.

4.2 How IaC aligns with the pipeline

The pipeline reads bucket names and AWS region from environment-specific YAML config files (e.g., `config/env.dev.yaml`). Terraform computes bucket names as:

`<project_name>-<environment>-<layer>`

which matches the repository defaults (e.g., `compos3d-dev-bronze`). The Glue crawler is explicitly pointed at the Silver prefix that the Silver stage writes (`compos3d/silver/`), so cataloging is consistent with the data lake layout.

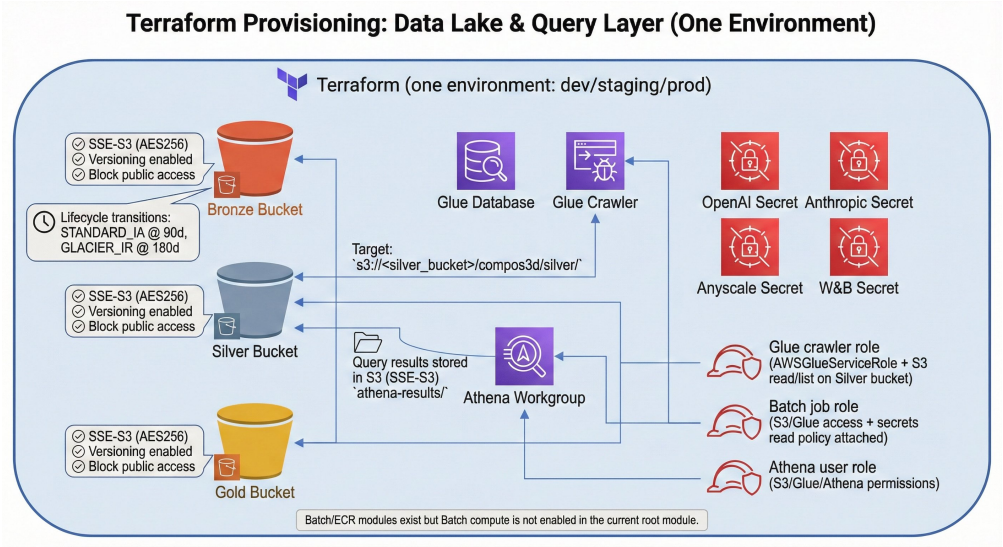

Figure 6: AWS resources provisioned by Terraform for one environment: S3 Bronze/Silver/Gold buckets; Glue database + crawler over Silver; Athena workgroup writing query results back to S3; Secrets Manager secret containers; IAM roles/policies.

4.3 Implementation of Multiple Environments

The infrastructure is parameterized by the `environment` variable, and the repository provides per-environment variable files under `terraform/environments/`. Each environment produces distinct bucket names and catalog names, so data and metadata are isolated by construction.

5 Disaster Recovery Demonstration

A screen-capture demonstration of the Infrastructure-as-Code (IaC) recovery process can be accessed at: YouTube Video

References

[1] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging ai applications, 2018. URL https://arxiv.org/abs/1712.05889.

[2] Shaofeng Yin, Jiaxin Ge, Zora Zhiruo Wang, Xiuyu Li, Michael J. Black, Trevor Darrell, Angjoo Kanazawa, and Haiwen Feng. Vision-as-inverse-graphics agent via interleaved multimodal reasoning, 2026. URL https://arxiv.org/abs/2601.11109.